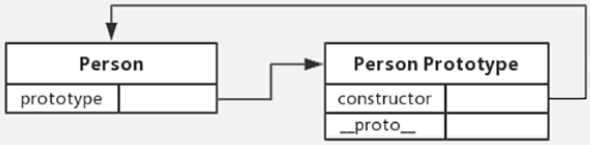
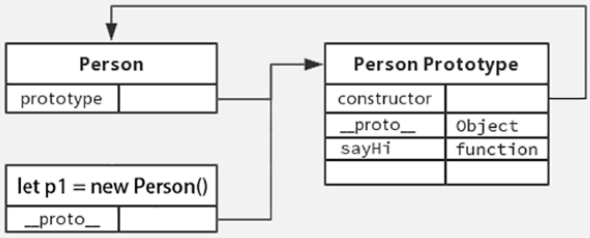
- **原型对象**
  - 每一个构造函数都有一个名为 prototype 的属性，称为构造函数的原型对象。每个原型对象都具有 constructor 属性代表了该原型对象对应的构造函数。

```
<script>
  function Person() {

  }

  // 每个函数都有 prototype 属性
  console.log(Person.prototype);
</script>
```



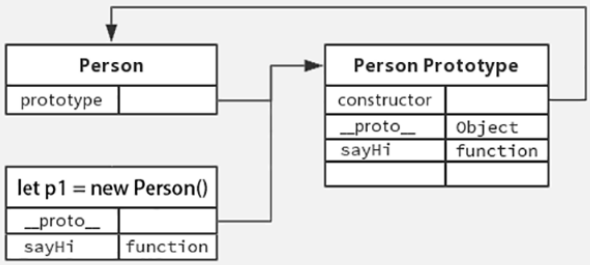
  - 了解了 JavaScript 中构造函数与原型对象的关系后，再来看原型对象具体的作用，如下代码所示：

```
<script>
  function Person() {
    // 此处未定义任何方法
  }

  // 为构造函数的原型对象添加方法
  Person.prototype.sayHi = function () {
    console.log('Hi~');
  }

  // 实例化
  let p1 = new Person();
  p1.sayHi(); // 输出结果为 Hi~
</script>
```



- 构造函数 `Person` 中未定义任何方法，这时实例对象调用了原型对象中的方法 `sayHi`，接下来改动一下代码：

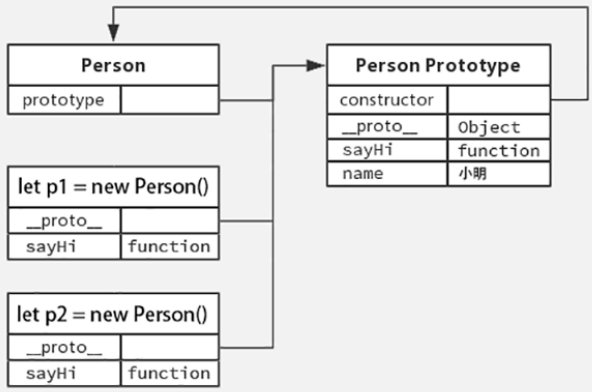
```
<script>
  function Person() {
    // 此处定义同名方法 sayHi
    this.sayHi = function () {
      console.log('嗨!');
    }
  }

  // 为构造函数的原型对象添加方法
  Person.prototype.sayHi = function () {
    console.log('Hi~');
  }

  let p1 = new Person();
  p1.sayHi(); // 输出结果为 嗨!
</script>
```



- 构造函数 `Person` 中定义与原型对象中相同名称的方法，这时实例对象调用则是构造函中的方法 `sayHi`。

- 通过以上两个简单示例不难发现 JavaScript 中对象的工作机制：“当访问对象的属性或方法时，先在当前实例对象是查找，然后再去原型对象查找，并且原型对象被所有实例共享。”

```
<script>
  function Person() {
    // 此处定义同名方法 sayHi
    this.sayHi = function () {
      console.log('嗨!' + this.name);
    }
  }

  // 为构造函数的原型对象添加方法
  Person.prototype.sayHi = function () {
    console.log('Hi~' + this.name);
  }
  // 在构造函数的原型对象上添加属性
  Person.prototype.name = '小明';

  let p1 = new Person();
  p1.sayHi(); // 输出结果为 嗨!小明

  let p2 = new Person();
  p2.sayHi(); // 输出结果为 嗨!小明
</script>
```



    - 原型对象是构造函数的一个属性，它的数据类型是对象
    - 原型对象对应的构造函数的实例方法或属性不存在时会去查找原型对象
  - 总结：结合构造函数原型的特征，实际开发重往往会将封装的功能函数添加到原型对象中。

- 继承
  - 继承是面向对象编程的另一个特征，通过继承进一步提升代码封装的程度，JavaScript 中大多是借助原型对象实现继承的特性。

- 龙生龙、凤生凤、老鼠的儿子会打洞描述的正是继承的含义，分别封装中国人和日本人的行为特征来理解编程中继承的含义，代码如下：

```
<script>
  // 封装中国人的行为特征
  function Chinese() {
    // 中国人的特征
    this.arms = 2;
    this.legs = 2;
    this.eyes = 2;

    this.skin = 'yellow';
    this.language = '中文';

    // 中国人的行为
    this.walk = function () {}
    this.sing = function () {}
    this.sleep = function () {}
  }

  // 封装日本人的行为特征
  function Japanese() {
    // 日本人的特征
    this.arms = 2;
    this.legs = 2;
    this.eyes = 2;

    this.skin = 'yellow';
    this.language = '日文';

    // 日本人的行为
    this.walk = function () {}
    this.sing = function () {}
    this.sleep = function () {}
  }
</script>
```

- 其实我们都知道无论是中国人、日本人还是其它民族，人们的大部分特征是一致的，然而体现在代码中时人的相同的行为特征被重复编写了多次，代码显得十分冗余，我们可以将重复的代码抽离出来。

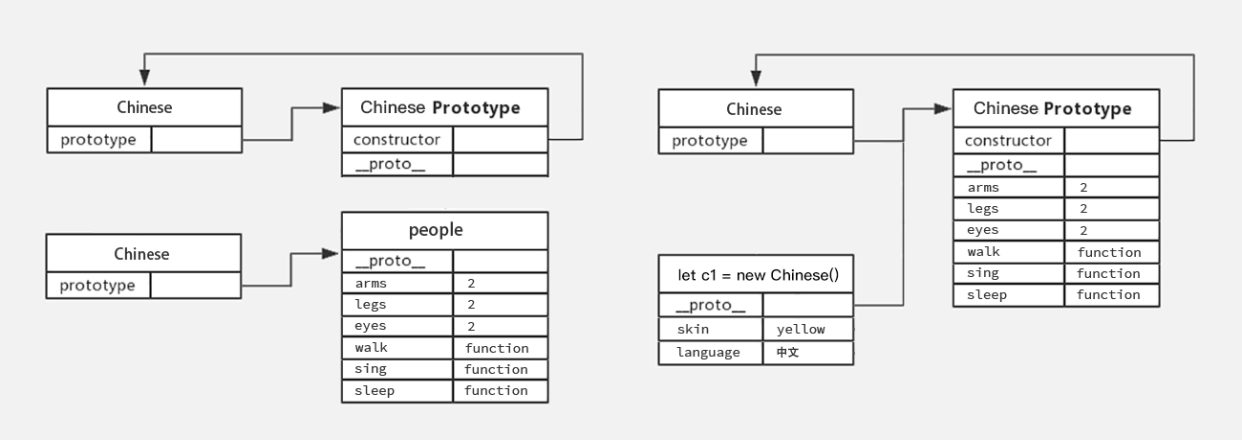
- 原型继承
  - 说明：基于构造函数原型对象实现面向对象的继承特性。

```
<script>
  // 所有人
  function Person() {
    // 人的特征
    this.arms = 2;
    this.legs = 2;
    this.eyes = 2;

    // 人的行为
    this.walk = function () {}
    this.sing = function () {}
    this.sleep = function () {}
  }

  // 中国人
  function Chinese() {
    this.skin = 'yellow';
    this.language = '中文';
  }
  // 日本人
  function Japanese() {
    this.skin = 'yellow';
    this.language = '日文';
  }
</script>
```

  - 上述代码可以理解成将 `Chinese` 和 `Japanese` 共有的属性和方法提取出来了，也就是说 `Chinese` 和 `Japanese` 需要【共享】一些属性和方法，而原型对象的属性和方法恰好是可以被用来共享的，因此我们看如下代码：

```
<script>
  // 中国人
  function Chinese() {
    this.skin = 'yellow';
    this.language = '中文';
  }
```

```
  // 日本人
  function Japanese() {
    this.skin = 'yellow';
    this.language = '日文';
  }

  // 人们【共有】的行为特征
  let people = {
    // 人的特征
    arms: 2,
    legs: 2,
    eyes:2,
    // 人的行为
    walk: function () {},
    sleep: function () {},
    sing: function () {}
  }

  // 为 prototype 重新赋值
  Chinese.prototype = people;
  Chinese.prototype.constructor = Chinese;
</script>
```



- 创建对象 `people` 将公共的的属性和方法独立出来，然后赋值给构造函数的 `prototype` 这样无论有多少个民族都可以共享公共的属性和方法了：

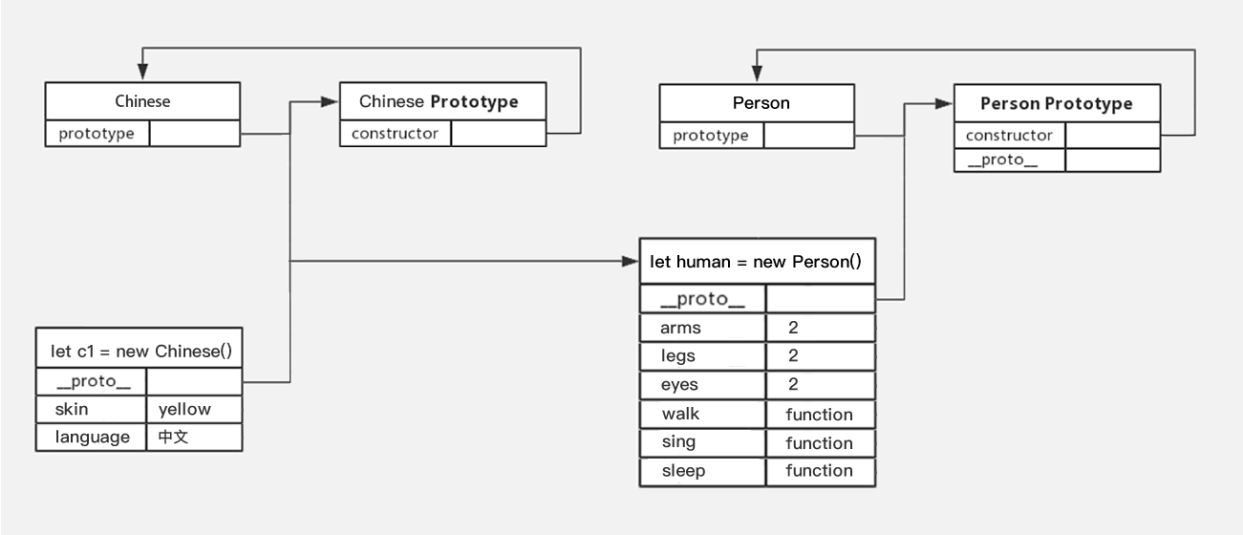
```
<script>
  // 人们【共有】的行为特征
  let people = {
    // 人的特征
    arms: 2,
```

```
    legs: 2,
    eyes:2,
    // 人的行为
    walk: function () {},
    sleep: function () {},
    sing: function () {}
  }

  // 中国人
  function Chinese() {
    this.skin = 'yellow';
    this.language = '中文';
  }
  // 日本人
  function Japanese() {
    this.skin = 'yellow';
    this.language = '日文';
  }

  function Englist() {
    this.skin = 'white';
    this.language= '英文';
  }

  // 中国人
  Chinese.prototype = people;
  Chinese.prototype.constructor = Chinese;

  let c1 = new Chinese();

  // 日本人
  Japanese.prototype = people;
  Janpanese.prototype.constructor = Japanese;
  // 英国人
  English.prototype = people;
  English.prototype.constructor = English;

  // ...
</script>
```

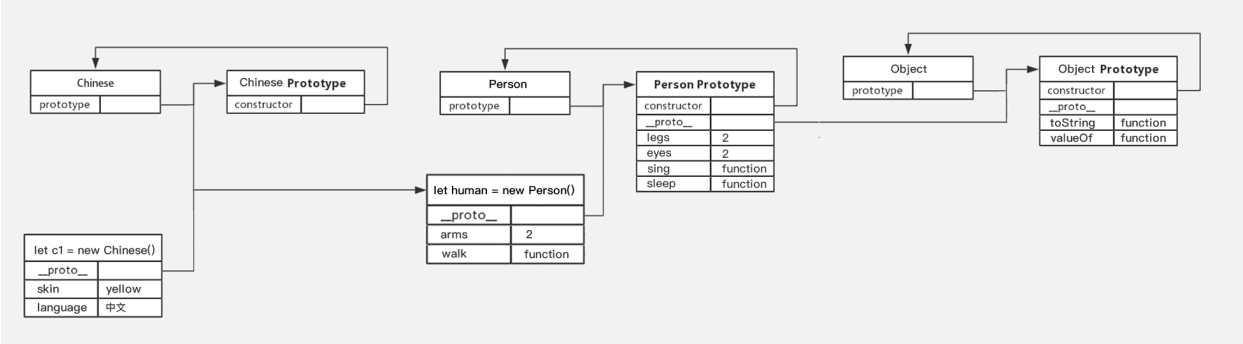
- 继承是一种可以“不劳而获”的手段，上述代码中 `Chinese`、`Japanese`、`English` 都轻松的获得了 `people` 的公共的方法和属性，我们说 `Chinese`、`Japanese`、`English` 继承了 `people`。
- 上述代码中是以命名空间的形式实现的继承，事实上 JavaScript 中继承更常见的是借助构造函数来实现：

```
<script>
  // 所有人
  function Person() {
    // 人的特征
    this.arms = 2;
    this.legs = 2;
    this.eyes = 2;

    // 人的行为
    this.walk = function () {}
    this.sing = function () {}
    this.sleep = function () {}
  }

  // 封装中国人的行为特征
  function Chinese() {
    // 中国人的特征
    this.skin = 'yellow';
    this.language = '中文';
  }

  // 封装日本人的行为特征
  function Japanese() {
    // 日本人的特征
    this.skin = 'yellow';
    this.language = '日文';
  }

  // human 是构造函数 Person 的实例
  let human = new Person();

  // 中国人
  Chinese.prototype = new Person();
  Chinese.prototype.constructor = Chinese;
  // 日本人
  Japanese.prototype = human;
  Japanese.prototype.constructor = Japanese;
```

```
</script>
```



- **原型链**
  - 基于原型对象的继承使得不同构造函数的原型对象关联在一起，并且这种关联的关系是一种链状的结构，我们将原型对象的链状结构关系称为原型链，如下图所示：




```
[]//作用：用于查找成员提供机制

<script>
  // Person 构造函数
  function Person() {
    this.arms = 2;
    this.walk = function () {}
  }

  // Person 原型对象
  Person.prototype.legs = 2;
  Person.prototype.eyes = 2;
  Person.prototype.sing = function () {}
```

```
  Person.prototype.sleep = function () {}

  // Chinese 构造函数
  function Chinese() {
    this.skin = 'yellow';
    this.language = '中文';
  }

  // Chinese 原型对象
  Chinese.prototype = new Person();
  Chinese.prototype.constructor = Chinese;

  // 实例化
  let c1 = new Chinese();
  console.log(c1);
</script>
```

- 在 JavaScript 对象中包括了一个非标准备的属性 __proto__ 它指向了构造函数的原型对象，通过它可以清楚的查看原型对象的链状结构。

- **异常**
  - 说明：异常处理是指预估代码执行过程中可能发生的错误，然后最大程度的避免错误的发生导致整个程序无法继续运行。
  - throw：

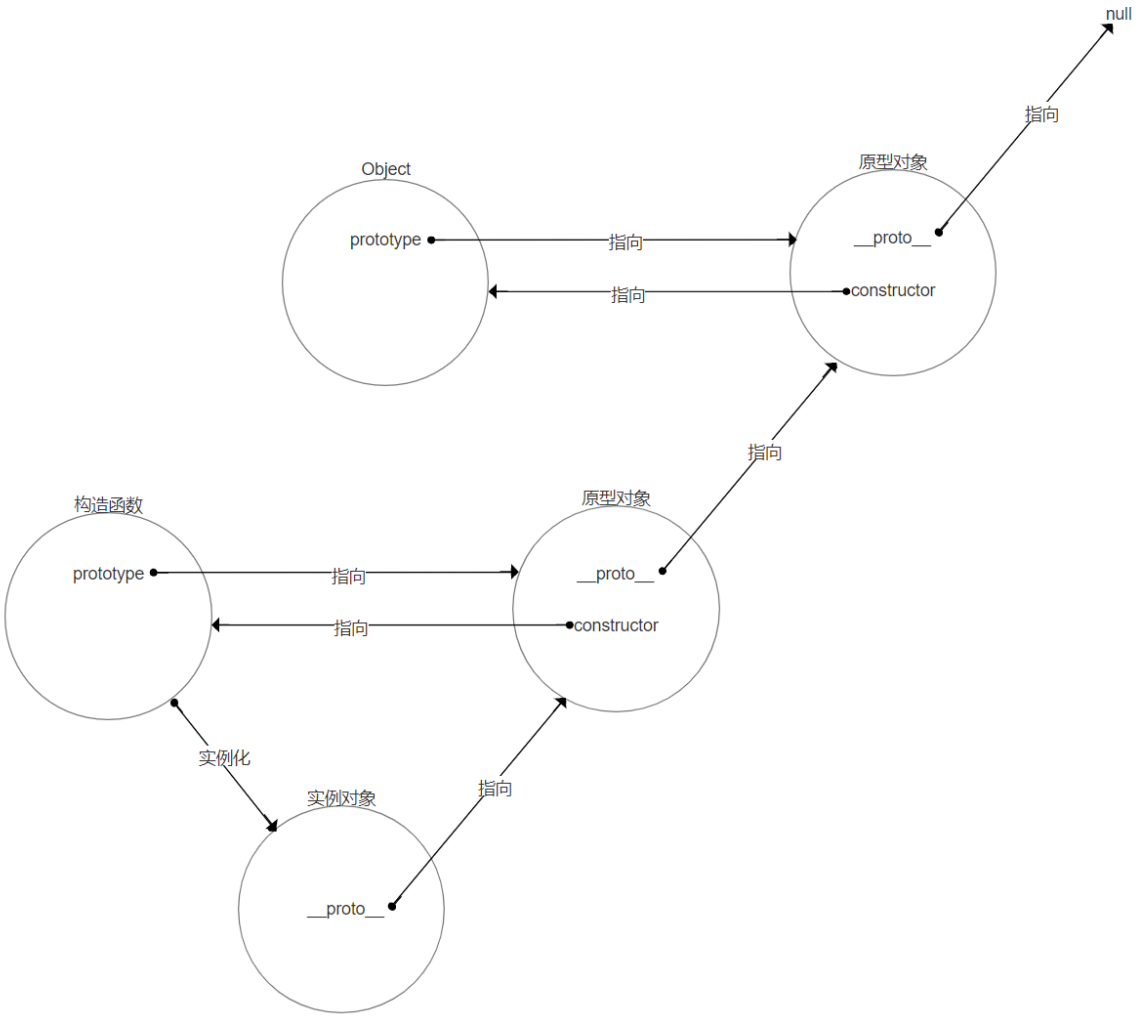
```
<script>
  function counter(x, y) {

    if(!x || !y) {
      // throw '参数不能为空!';
      throw new Error('参数不能为空!');
    }

    return x + y;
  }

  counter();
</script>
```

        - throw 抛出异常信息，程序也会终止执行
        - throw 后面跟的是错误提示信息
        - Error 对象配合 throw 使用，能够设置更详细的错误信息
    - try ... catch：

```
<script>
  function foo() {

    try {
      // 查找 DOM 节点
      var p = docunent.querySelector('p');

    } catch(error) {
      // try 代码段中执行有错误时，会执行 catch 代码段

      // 查看错误信息
      console.log(error.message);

      // 终止代码继续执行
      return;
    }

    // 改变文本样式
    p.style.color = 'red';
  }

  foo();
</script>
```

        - try...catch 用于捕获错误信息
        - 将预估可能发生错误的代码写在 try 代码段中
        - 如果 try 代码段中出现错误后，会执行 catch 代码段，并截获到错误信息

- **原型总结图**


null
指向
Object
原型对象
prototype
指向
__proto__
指向
constructor
指向
构造函数
原型对象
prototype
指向
__proto__
指向
constructor
实例化
实例对象
指向
__proto__